TEK-UP – École Supérieure Privée de Technologie & d'Ingénierie

Module: OWASP – Académique 2025 / 2026

OWASP JUICE SHOP

Web Application Security Assessment – Findings Report

Business Confidential

Target: http://juiceshop.com:3000

Prepared by: Ahmed Rafraf

ING-4-J-SSIRF-H

Date: 05/06/2026

Version 1.0

Table of Contents

Confidentiality Statement 3
Disclaimer 3
Contact Information 3
Assessment Overview 4
Assessment Component – Web Application Penetration Test 4
Finding Severity Ratings 5
Risk Factors 5
Scope 6
Scope Exclusions 6
Executive Summary 7
Testing Summary 7
Key Weaknesses 7
Vulnerability Summary & Report Card 8
Technical Findings 9
Finding WAPT-001: SQL Injection – Authentication Bypass (Critical) 10
Finding WAPT-002: Improper Input Validation – Negative Basket Quantity (Business Logic) (High) 12
Finding WAPT-003: Broken Access Control – IDOR on Shopping Basket (High) 14
Finding WAPT-004: Cross-Site Scripting (XSS) – DOM-based in Search (High) 16
Finding WAPT-005: Sensitive Data Exposure – Confidential Files & Directory Access (High) 18
Finding WAPT-006: Cryptographic Failures – Weak Password Storage (High) 21
Finding WAPT-007: Mass Assignment – Privilege Escalation at Registration (High) 24
Finding WAPT-008: Security Misconfiguration – Verbose Errors & Information Disclosure (Moderate) 26
Appendix A – Methodology & Tools 28

Confidentiality Statement

This document contains proprietary and confidential information relating to a security assessment performed against an instance of OWASP Juice Shop for educational purposes within the OWASP module at TEK-UP. It is intended solely for the assessment team and the module supervisors. Duplication or redistribution requires the consent of the author.

Disclaimer

A penetration test is a snapshot in time. The findings and recommendations reflect information gathered during the assessment and not any changes made afterwards. OWASP Juice Shop is an intentionally insecure application published by OWASP for training; all testing described here was conducted on a local instance owned and operated by the tester, with no impact on third parties.

Contact Information

Name	Role	Contact
Ahmed Rafraf	Student / Tester	ahmedrafraf36@gmail.com
Ahmed Snoussi	Module Supervisors	TEK-UP – OWASP Module

Assessment Overview

This assessment evaluated the security posture of the OWASP Juice Shop web application running locally at http://localhost:3000. Testing followed an industry-standard methodology aligned with the OWASP Web Security Testing Guide (WSTG) and the OWASP Top 10 (2021), supplemented by manual exploitation. The objectives, as defined by the module brief, were to become familiar with web vulnerabilities and to perform attacks that identify and exploit them.

The penetration testing activities followed four phases:

1. **Planning** – Define objectives, scope and rules of engagement.
2. **Discovery** – Reconnaissance, fingerprinting and enumeration to identify potential weaknesses.
3. **Attack** – Confirm vulnerabilities through controlled exploitation and pivot to new findings.
4. **Reporting** – Document findings, evidence, impact and remediation.

Assessment Component – Web Application Penetration Test

A web application penetration test emulates the role of an external or authenticated attacker interacting with the application. The tester enumerates the attack surface, identifies vulnerabilities such as injection, broken access control and cryptographic failures, and seeks to exploit them to demonstrate impact – for example account takeover, data disclosure and privilege escalation.

Finding Severity Ratings

The following table defines the severity levels and corresponding CVSS v3 score ranges used throughout this report.

Severity	CVSS v3 Range	Definition
Critical	9.0 – 10.0	Exploitation is straightforward and usually results in system-level compromise. Form a plan of action and patch immediately.
High	7.0 – 8.9	Exploitation is more difficult but could cause elevated privileges, loss of data or downtime. Patch as soon as possible.
Moderate	4.0 – 6.9	Vulnerabilities exist but are harder to exploit or require extra steps. Patch after high-priority issues.
Low	0.1 – 3.9	Non-exploitable issues that would reduce attack surface. Patch during the next maintenance window.
Informational	N/A	No vulnerability. Information noted during testing, strong controls, or additional documentation.

Risk Factors

Risk is measured by two factors: Likelihood and Impact.

Likelihood measures the potential of a vulnerability being exploited, based on the difficulty of the attack, available tooling, attacker skill and the environment.

Impact measures the effect on confidentiality, integrity and availability of the application and its data, as well as reputational and operational harm.

Scope

The assessment was limited to the locally hosted OWASP Juice Shop instance:

- Target application: OWASP Juice Shop
- URL: http://localhost:3000
- Environment: Kali Linux (local lab)

Test environment note: To intercept and tamper with requests in Burp Suite, the hostname juiceshop.com was mapped to 127.0.0.1 in the local hosts file, because the browser and proxy bypass localhost traffic by default. The application was therefore reached at http://juiceshop.com:3000, which is the same local instance as http://localhost:3000. This is why some request captures in this report show the juiceshop.com host.



The local hosts file showing the juiceshop.com → 127.0.0.1 entry used for interception.

Scope Exclusions

Denial-of-Service attacks and attacks against any system other than the local Juice Shop instance were out of scope. No real users or third-party systems were involved.

Executive Summary

This report summarises the vulnerabilities discovered during a web application penetration test of the OWASP Juice Shop. The assessment identified a range of issues spanning most of the OWASP Top 10 (2021) categories, several of which allow complete compromise of the application and its data.

Testing Summary

Reconnaissance established that the target is a Node.js / Express backend serving an Angular single-page application, and the hidden Score Board was used to guide testing. The most serious finding was a SQL injection in the login form: an authentication-bypass payload granted immediate administrator access without any valid credentials.

Broken access control allowed shopping baskets to be read across accounts through an insecure direct object reference (IDOR), and weak server-side input validation let the basket total be driven negative by tampering with item quantities. Further findings included DOM-based cross-site scripting in the search feature, exposure of confidential files through a browsable directory, weak (unsalted MD5) password storage, privilege escalation through mass assignment at registration, and security misconfiguration in the form of verbose error messages.

Collectively, these findings demonstrate that an attacker requires little skill or tooling to fully compromise the application, exfiltrate all customer data and obtain administrative control.

Key Weaknesses

1. User input is not parameterized, enabling SQL injection.
2. Access control is enforced only on the client side, allowing cross-account basket reads (IDOR), and numeric inputs such as basket quantity are not validated on the server.
3. Output is not encoded, enabling cross-site scripting.
4. Confidential files are exposed through a browsable directory.
5. Passwords are stored using a weak, unsalted hash (MD5).
6. The registration endpoint allows privilege escalation through mass assignment.
7. Verbose error messages disclose internal application detail.

Vulnerability Summary & Report Card

The following table summarises the vulnerabilities found by severity, followed by the per-finding report card.

1	6	1	0	0
Critical	**High**	**Moderate**	**Low**	**Informational**

Finding	Severity	Recommendation
WAPT-001: SQL Injection – Authentication Bypass	**Critical**	Use parameterized queries and input validation.
WAPT-002: Improper Input Validation – Negative Basket Quantity (Business Logic)	**High**	Validate quantities server-side (positive, bounded); recompute totals from trusted data.
WAPT-003: Broken Access Control – IDOR on Shopping Basket	**High**	Add object-level access checks (ownership).
WAPT-004: Cross-Site Scripting (XSS) – DOM-based in Search	**High**	Encode output; apply a strict CSP.
WAPT-005: Sensitive Data Exposure – Confidential Files & Directory Access	**High**	Remove files from web root; disable listing.
WAPT-006: Cryptographic Failures – Weak Password Storage	**High**	Hash passwords with bcrypt/Argon2 and salt.
WAPT-007: Mass Assignment – Privilege Escalation at Registration	**High**	Allow-list bindable fields; set roles server-side.
WAPT-008: Security Misconfiguration – Verbose Errors & Information Disclosure	**Moderate**	Generic errors; restrict operational endpoints.

Technical Findings

Each finding below includes a description, OWASP Top 10 (2021) mapping, risk rating, affected component, tools used, references, an evidence placeholder for your screenshots, and remediation guidance. Replace each evidence box with a screenshot captured on your own Juice Shop instance, and confirm the finding before submission.

Finding WAPT-001: SQL Injection – Authentication Bypass (Critical)

Description:	The login form does not parameterize the email field before building the backend SQL query. By submitting a crafted value in the email field, an attacker is able to bypass authentication completely and log in as the first record in the Users table, which is the administrator account. Proof-of-concept payload (email field): `' or 1=1 -- -` With any value in the password field, the application returns a valid session (JWT) for the administrator, granting full access to administrative functionality and all customer data.
OWASP Category:	A03:2021 – Injection
Risk:	**Likelihood:** High – Exploitable directly from the browser with a single well-known payload; no tooling required. **Impact:** Very High – Complete administrative account takeover, exposing all user data and privileged functions.
Affected Component:	POST /rest/user/login (email parameter) – http://localhost:3000/#/login
Tools Used:	Web browser, Burp Suite, sqlmap
References:	OWASP Top 10 2021 – A03 Injection OWASP WSTG – Testing for SQL Injection (WSTG-INPV-05) OWASP SQL Injection Prevention Cheat Sheet

Evidence

Testing started by confirming how the login form behaves with ordinary input, then the email field was probed for SQL injection before submitting the bypass payload.

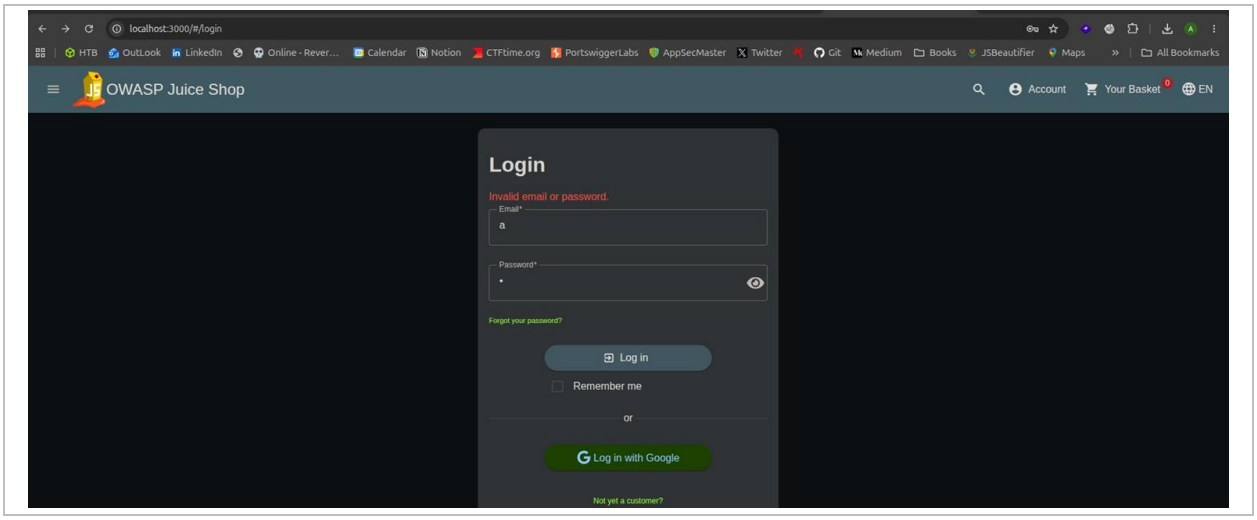


Figure 1: A normal login attempt with invalid credentials returns the standard "Invalid email or password" message – the expected baseline behaviour.

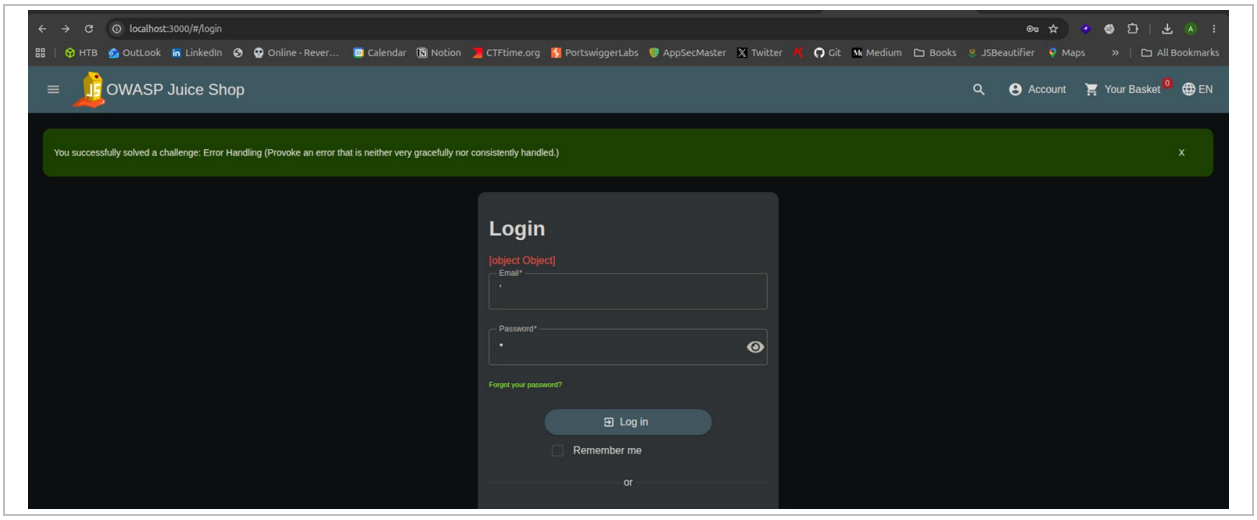


Figure 2: Submitting a single quote (') in the email field breaks the underlying SQL query, and the application responds with an unhandled "[object Object]" error. This confirms the input reaches the database layer and also triggers the improper error handling noted in WAPT-008.

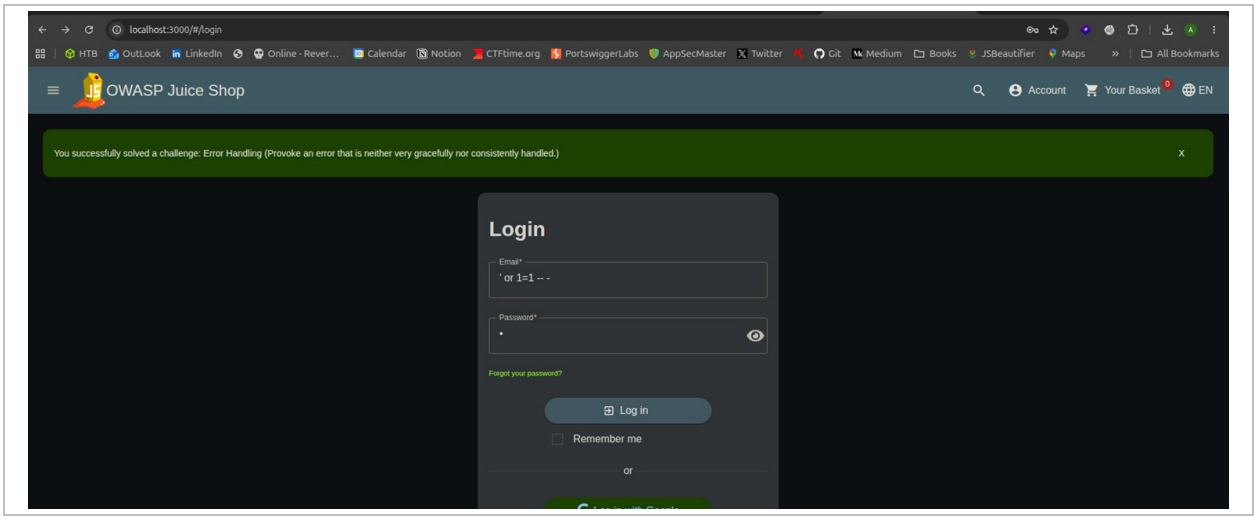


Figure 3: The injection payload ' or 1=1 -- - placed in the email field bypasses the authentication check, returning a valid session without knowing any password.

Remediation

Use parameterized queries / prepared statements for every database interaction and rely on the ORM's parameter binding (Sequelize) rather than string concatenation.

Apply strict server-side input validation, run the database account with least privilege, and add a Web Application Firewall as a defence-in-depth control.

Finding WAPT-002: Improper Input Validation – Negative Basket Quantity (Business Logic) (High)

Description:	The shopping basket sets item quantities through the API, but the server does not validate that the quantity is a positive number. The browser only enforces this on the client side, so by intercepting the add-to-basket request and submitting a negative quantity, the line total and the basket total are driven negative. This is a business-logic / improper input validation flaw: the application trusts client-supplied numbers and performs no server-side bounds checking. Tampered request: **`POST /api/BasketItems with "quantity": -10`** The negative total is not rejected: the basket displays a negative Total Price and the Checkout button stays enabled, so an order whose total is in the customer's favour can be carried through to checkout rather than the shop's.
OWASP Category:	A04:2021 – Insecure Design (Improper Input Validation)
Risk:	**Likelihood:** High – A single intercepted request with a tampered numeric value; no special tooling beyond an intercepting proxy. **Impact:** High – An order can be checked out with a negative total, so the shop effectively pays the customer: direct financial loss and a broken purchasing process.
Affected Component:	POST/PUT /api/BasketItems (quantity parameter)
Tools Used:	Burp Suite, web browser
References:	OWASP Top 10 2021 – A04 Insecure Design OWASP Input Validation Cheat Sheet

Evidence

The browser refuses negative quantities in the basket UI, but the server applies no such check. Intercepting the add-to-basket request and replacing the quantity with a negative number is accepted, and the negative value flows through to the basket total and the checkout step.

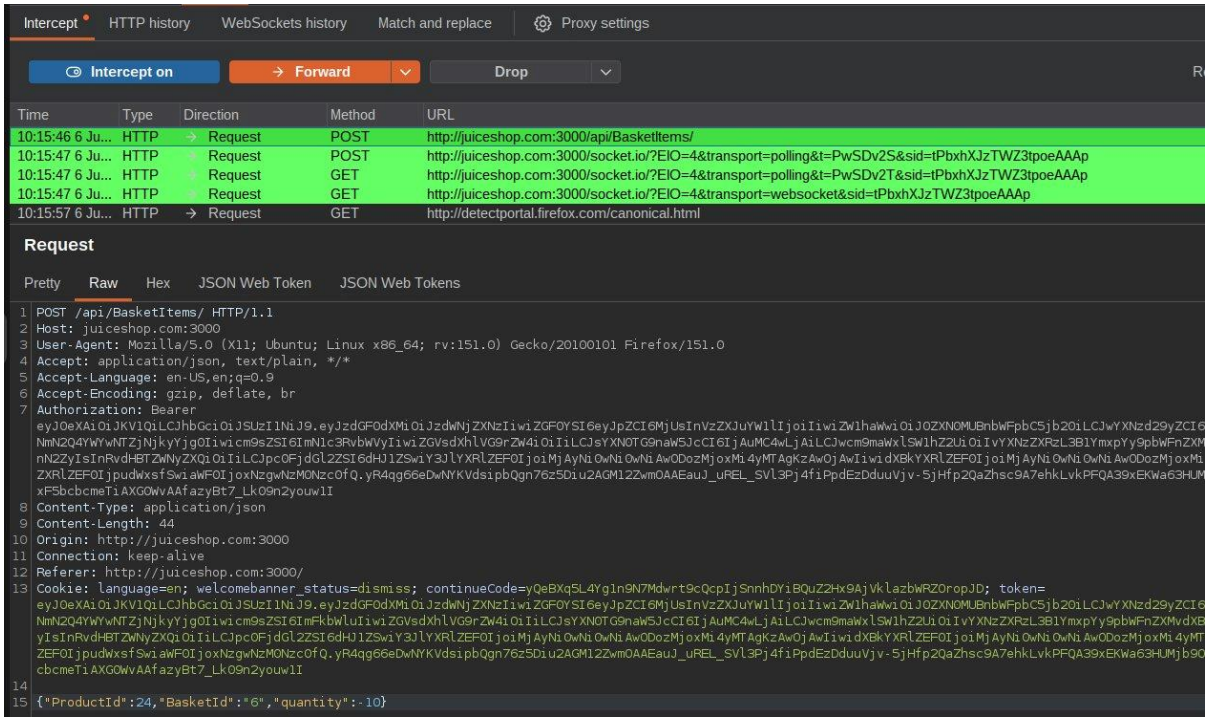


Figure 4a: Step 1 – Tampering the quantity. The add-to-basket request POST /api/BasketItems is intercepted in Burp Suite and the JSON body is changed to a negative quantity — {"ProductId":24,"BasketId":"6","quantity":-10}. The server accepts the request; there is no server-side check that the quantity must be positive.

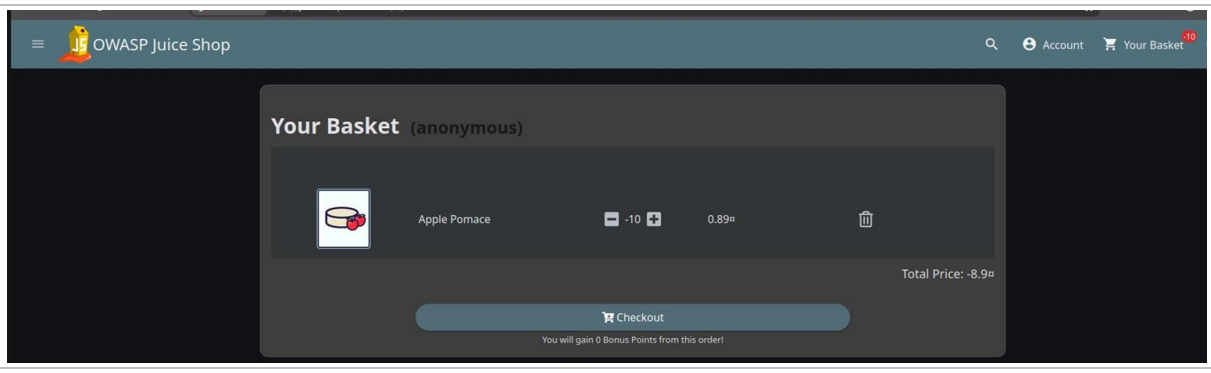


Figure 4b: Step 2 – Negative total carried to checkout. The basket now holds a quantity of -10 for Apple Pomace and the Total Price is computed as -8.9 (10 × 0.89, negated). The negative order is not blocked: the Checkout button stays enabled and the app reports "You will gain 0 Bonus Points from this order!" — a customer could place an order whose total is in their own favour.

Remediation

Validate all numeric inputs on the server: enforce that quantity is a positive integer within sensible bounds, and reject anything else with a 4xx error.

Never trust client-side validation alone, and always recompute order totals on the server from trusted product data before accepting payment.

Finding WAPT-003: Broken Access Control – IDOR on Shopping Basket (High)

Description:	Basket retrieval and modification endpoints use a predictable numeric identifier and do not verify that the basket belongs to the authenticated user. This is an Insecure Direct Object Reference (IDOR). Vulnerable endpoint: **`GET /rest/basket/{id}`** By changing the basket id (or editing the BasketId value in the request when adding an item), an attacker can view and manipulate the contents of other customers' baskets.
OWASP Category:	A01:2021 – Broken Access Control
Risk:	**Likelihood:** High – Identifiers are sequential and the request is simple to tamper with using an intercepting proxy. **Impact:** High – Unauthorized read and modification of other users' data.
Affected Component:	GET/PUT /rest/basket/{id}, POST /api/BasketItems
Tools Used:	Burp Suite
References:	OWASP Top 10 2021 – A01 Broken Access Control OWASP WSTG – Testing for IDOR (WSTG-ATHZ-04)

Evidence

Logged in to the Admin account, the "Your Basket" page was opened while Burp Suite intercepted the traffic, so the basket request could be captured. The basket identifier in that request was then changed from 1 to 2 and forwarded – and it worked, leaking another user's basket information.

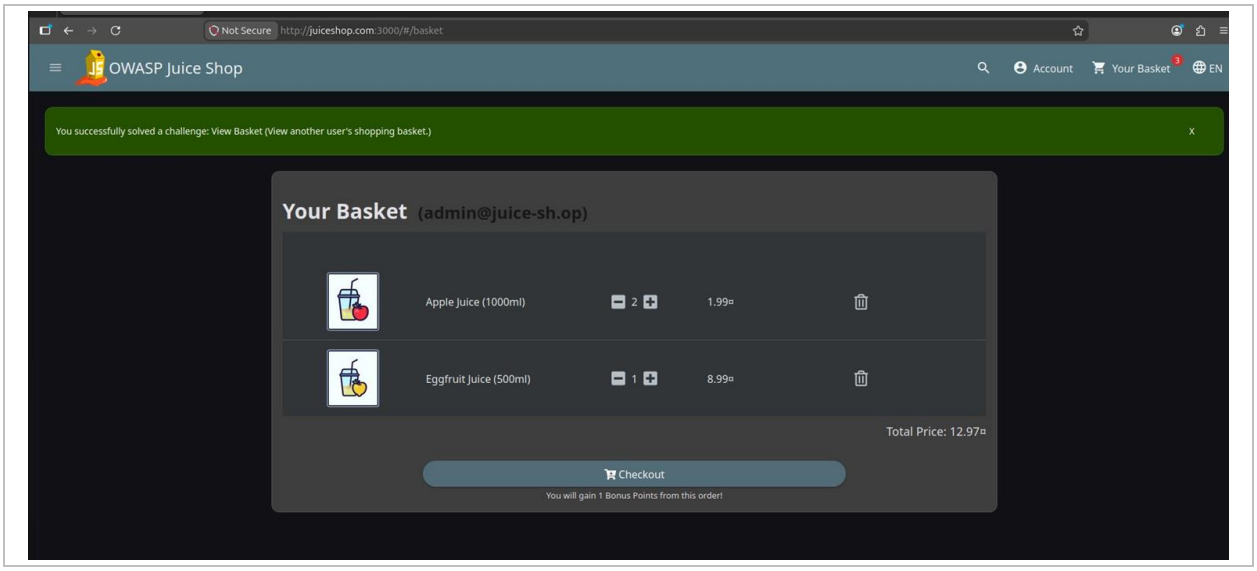


Figure 5: The "Your Basket" page opened in the browser. Loading this page is what sends the basket request that is intercepted in Burp Suite in the steps below.

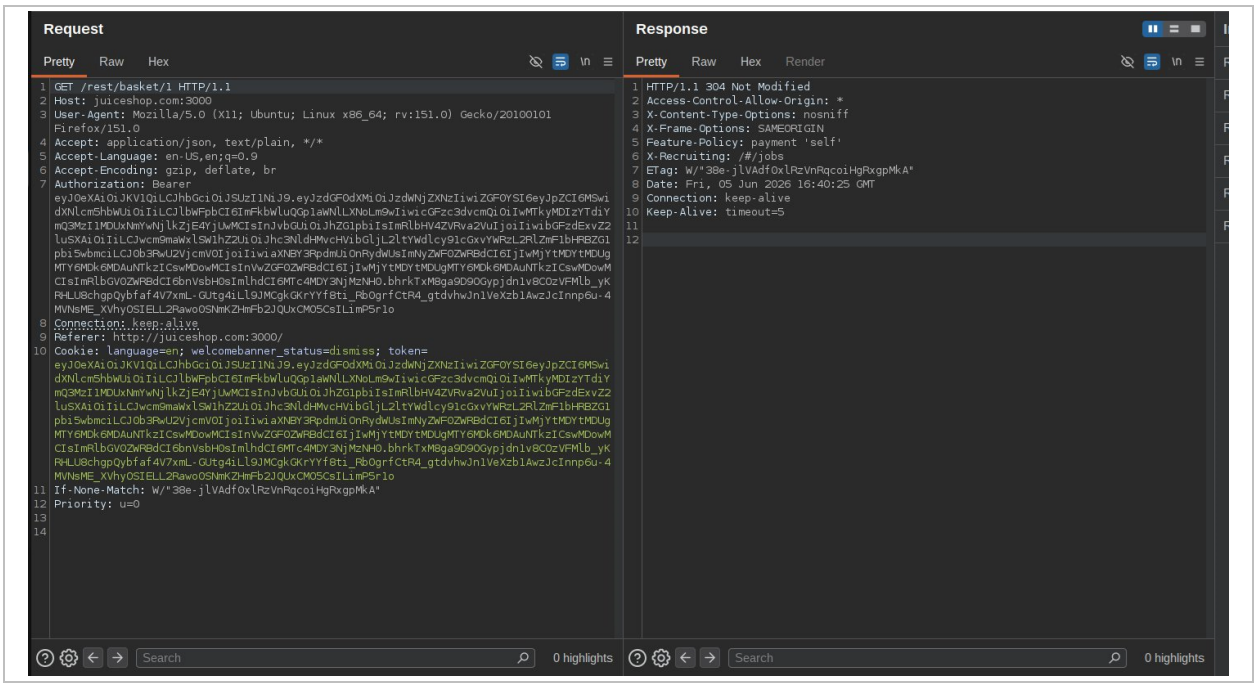


Figure 6: The intercepted request in Burp Suite: GET /rest/basket/1, which retrieves the logged-in (Admin) user's own basket.

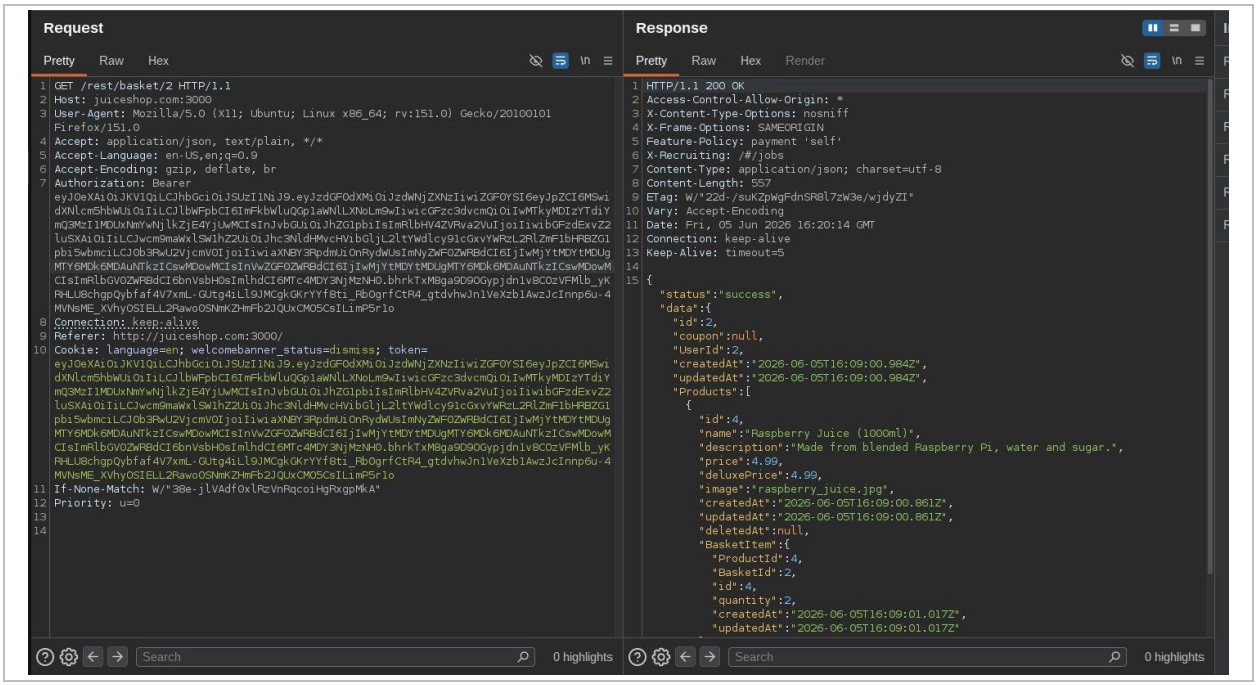


Figure 7: Changing the identifier in the path from 1 to 2 (GET /rest/basket/2) and forwarding the request returns 200 OK with another user's basket contents – products, quantities and the owning UserId – with no ownership check performed by the server.

Remediation

Implement object-level authorization that ties each resource to the session user, and reject requests for objects the user does not own. Avoid exposing sequential, predictable identifiers; consider indirect references mapped per session.

Finding WAPT-004: Cross-Site Scripting (XSS) – DOM-based in Search (High)

Description:	The search-results header reflects the query string into the page DOM without sanitization, resulting in a DOM-based Cross-Site Scripting vulnerability. Arbitrary JavaScript executes in the victim's browser context. Payload (search box / q parameter): **`<iframe src="javascript:alert(`xss`)">`** An attacker can leverage this to steal session tokens (JWT), perform actions as the victim, or deface the application.
OWASP Category:	A03:2021 – Injection (Cross-Site Scripting)
Risk:	**Likelihood:** High – Trivially triggered through the search field with a public payload. **Impact:** High – Session token theft and actions performed in the victim's context.
Affected Component:	Search functionality – http://localhost:3000/#/search?q=
Tools Used:	Web browser, Burp Suite
References:	OWASP Top 10 2021 – A03 Injection OWASP XSS Prevention Cheat Sheet

Evidence

The search field reflects the query back into the page, so an HTML/JavaScript payload was supplied to confirm that script executes in the browser context.

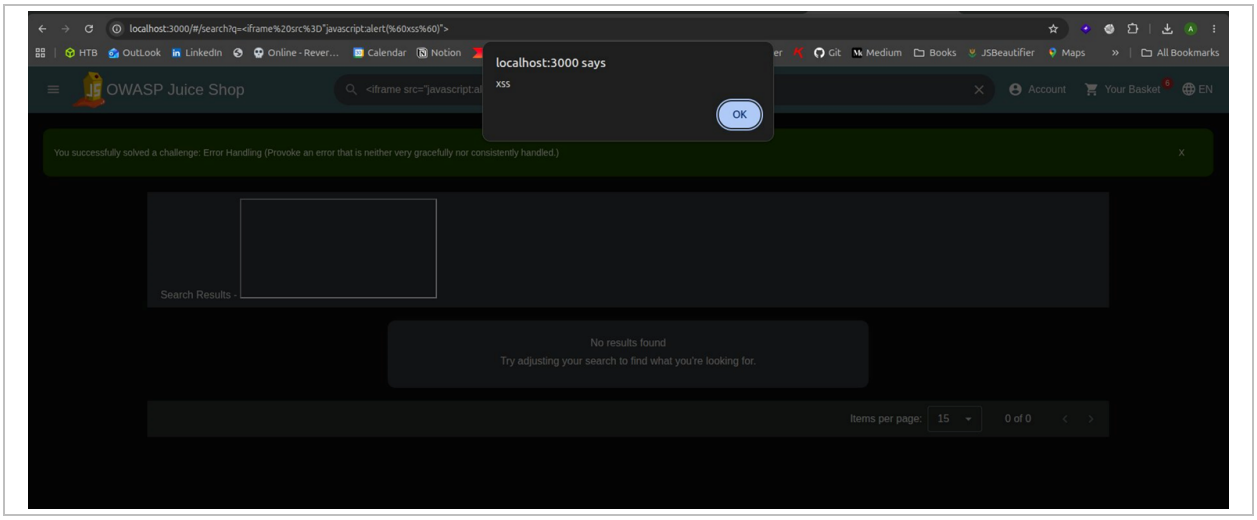


Figure 8: The payload <iframe src="javascript:alert(`xss`)"> entered through the search bar executes JavaScript in the browser and raises the "xss" alert dialog, confirming a DOM-based XSS vulnerability.

Remediation

Apply contextual output encoding and rely on the framework's built-in escaping; never bind untrusted input into the DOM as HTML.

Deploy a strict Content Security Policy (CSP) and sanitize any HTML that must be rendered.

Finding WAPT-005: Sensitive Data Exposure – Confidential Files & Directory Access (High)

Description:	The static /ftp directory is browsable and exposes confidential files, including an internal acquisition memo. Direct download of non-permitted file types is restricted by an extension filter, but the filter is bypassable using a poison null byte. Browsable directory: **`http://localhost:3000/ftp`** Filter bypass example: **`http://localhost:3000/ftp/package.json.bak%2500.md`**
OWASP Category:	A01:2021 – Broken Access Control / A02:2021 – Cryptographic Failures
Risk:	**Likelihood:** High – Directory listing is reachable without authentication. **Impact:** High – Disclosure of confidential business documents and internal configuration backups.
Affected Component:	http://localhost:3000/ftp
Tools Used:	Web browser
References:	OWASP Top 10 2021 – A01 Broken Access Control OWASP WSTG – Review Old Backup and Unreferenced Files (WSTG-CONF-04)

Evidence

The static /ftp directory is served without authentication and lists its own contents in the browser. Browsing it exposed several confidential files: one business memo opened directly because its extension was allowed, a backup file was blocked by an extension filter, and that filter was then bypassed using a poison null byte.

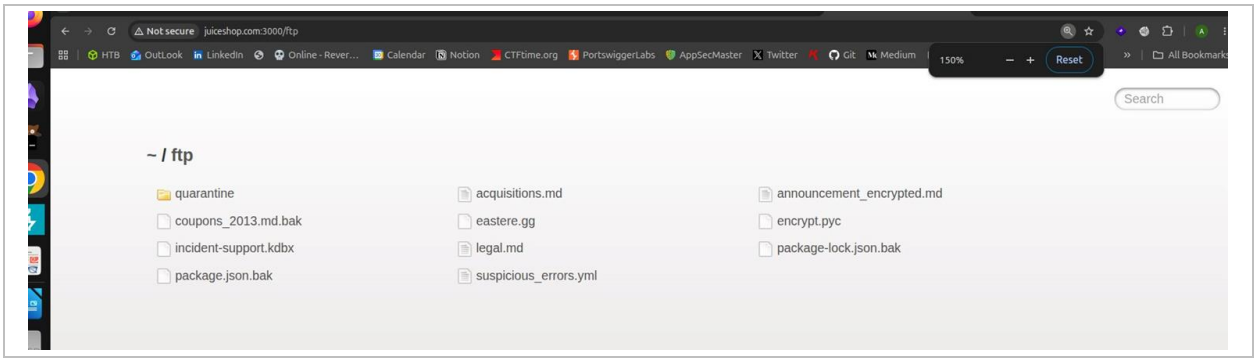


Figure 9a: Step 1 – Browsing the directory. Requesting /ftp returns a full directory listing with no authentication required. Several sensitive files are exposed, including acquisitions.md, package.json.bak, package-lock.json.bak, incident-support.kdbx and suspicious_errors.yml.

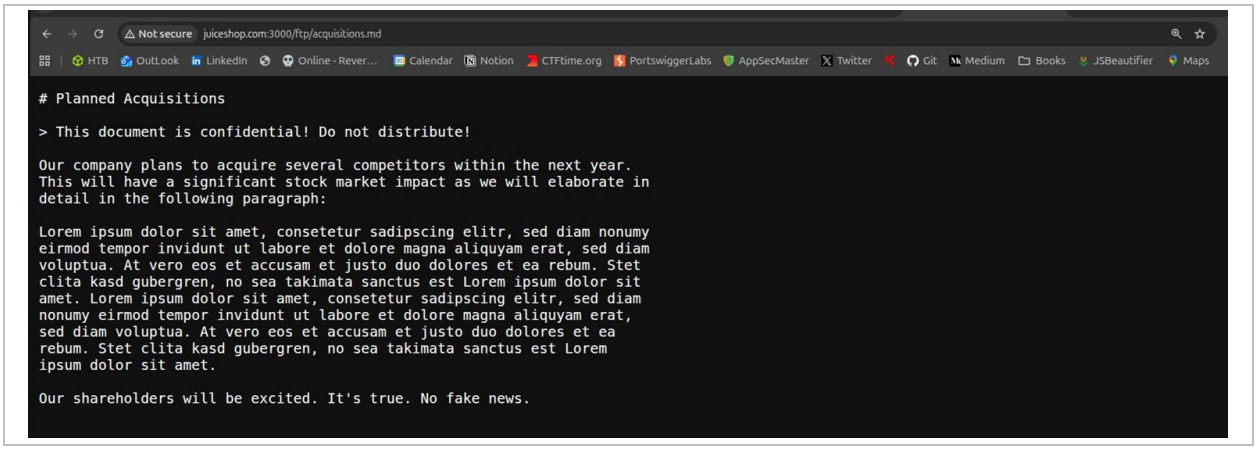


Figure 9b: Step 2 – Reading a confidential file. Because .md is an allowed extension, acquisitions.md opens directly in the browser. It is an internal memo explicitly marked "This document is confidential! Do not distribute!" describing planned competitor acquisitions — a clear exposure of business data.

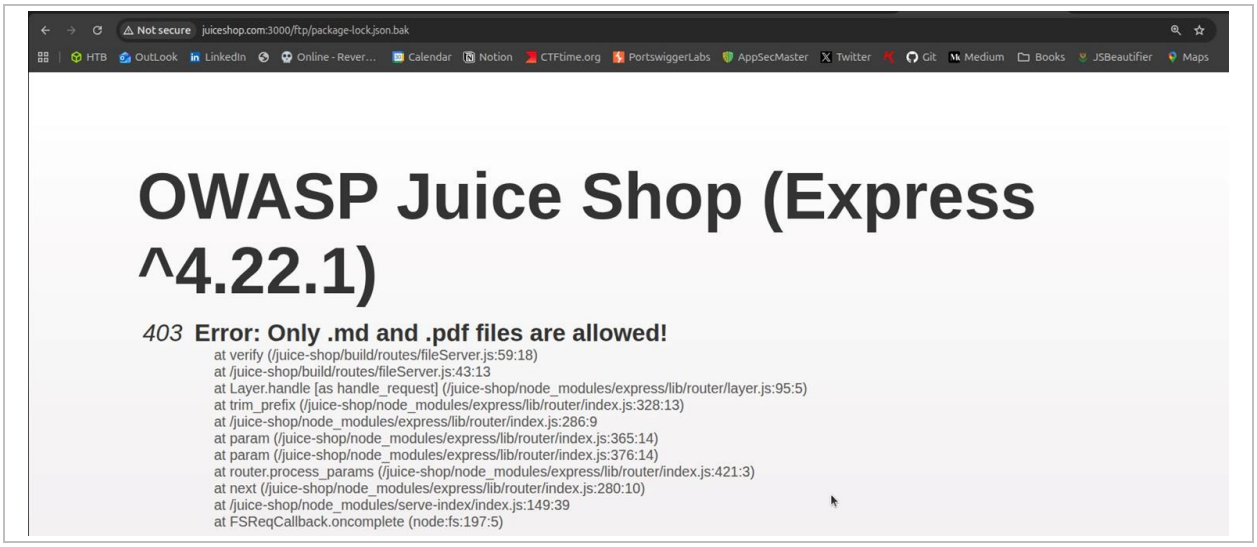


Figure 9c: Step 3 – The extension filter. Requesting a non-permitted file such as package-lock.json.bak directly is rejected with 403 "Only .md and .pdf files are allowed!". The verbose stack trace returned alongside the error also leaks internal server file paths (see WAPT-008).

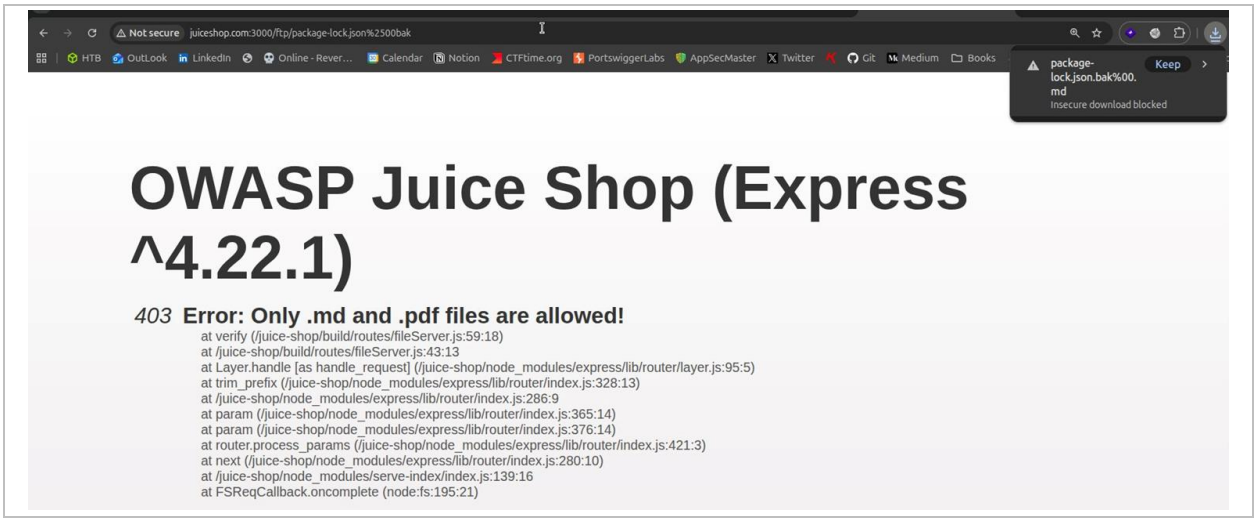


Figure 9d: Step 4 – Bypassing the filter with a poison null byte. Requesting the backup file with an encoded null byte followed by an allowed extension (package-lock.json.bak%2500.md) defeats the check: the server reads the name only up to the null byte and serves the .bak file. The browser's download prompt for package-lock.json.bak%00.md confirms the restricted file was returned — the download was blocked only because the site is served over plain HTTP.

Remediation

Remove sensitive files from the web root, disable directory listing, and store backups outside any web-accessible path.

Validate file paths and extensions robustly; reject encoded characters such as null bytes.

Finding WAPT-006: Cryptographic Failures – Weak Password Storage (High)

Description:	User credentials are stored as unsalted MD5 hashes. After extracting the Users table through a UNION-based SQL injection in the product-search endpoint, the recovered hashes were cracked within seconds using public lookup tables (CrackStation), revealing plaintext passwords, including those of administrative accounts. MD5 is a fast, broken hashing algorithm unsuitable for password storage; the absence of a per-user salt allows pre-computed (rainbow table) attacks.
OWASP Category:	A02:2021 – Cryptographic Failures
Risk:	**Likelihood:** High – Once hashes are obtained, cracking is fast and inexpensive. **Impact:** High – Plaintext recovery of user and administrator credentials.
Affected Component:	Users table – password column
Tools Used:	Burp Suite, CrackStation
References:	OWASP Top 10 2021 – A02 Cryptographic Failures OWASP Password Storage Cheat Sheet

Evidence

Separately from the login form (WAPT-001), the product-search endpoint (GET /rest/products/search?q=) was also found to be injectable. A UNION-based SQL injection here allowed the database to be read directly, and this is how the Users table and the password hashes stored in it were extracted. The steps below reconstruct that extraction and the offline cracking that followed.

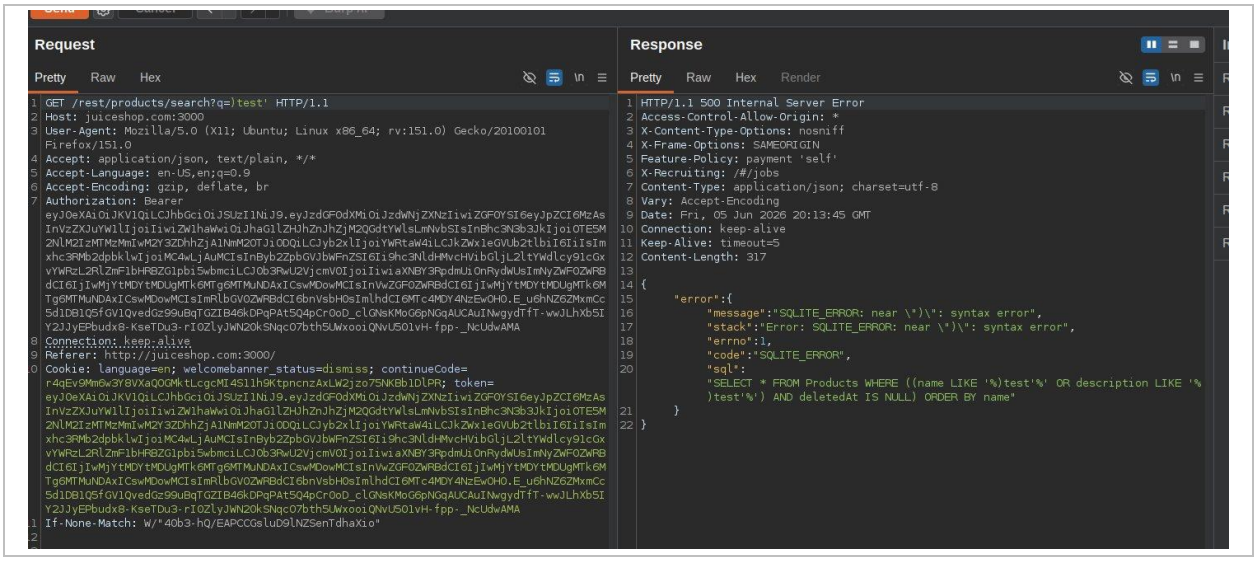


Figure 10a: Step 1 – Confirming the injection point. A single quote in the search query (q=)test') breaks the SQL statement, and the server returns a verbose SQLITE_ERROR containing the raw query. This proves three things at once: the q parameter is injectable, the backend database is SQLite, and the supplied text (test) is concatenated

straight into the WHERE clause.

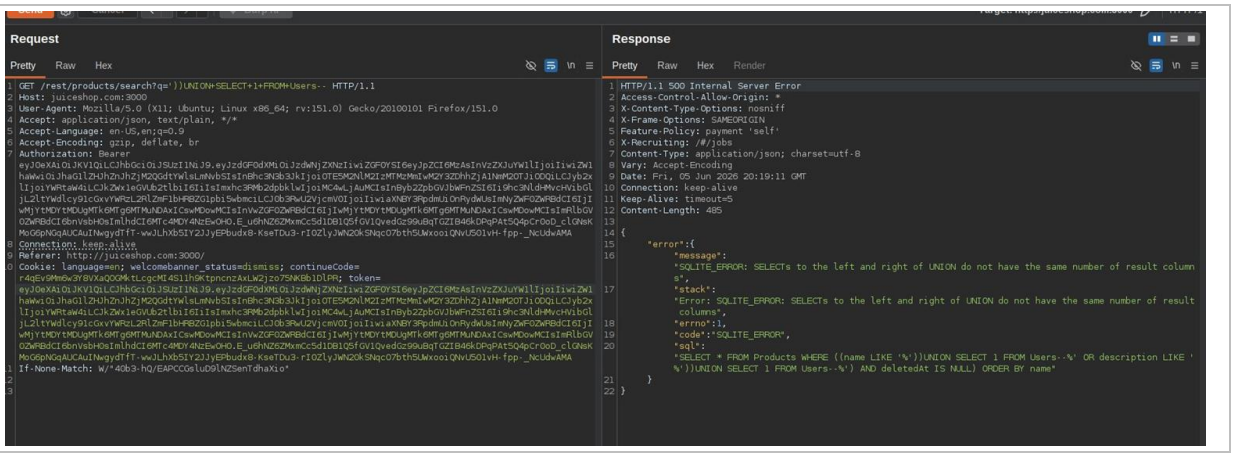


Figure 10b: Step 2 – Finding the number of columns. The first UNION attempt ('')) UNION SELECT 1 FROM Users--) fails with "SELECTs to the left and right of UNION do not have the same number of result columns." A UNION only works when both queries return the same number of columns, so the original query must return more than one — the count has to be matched.

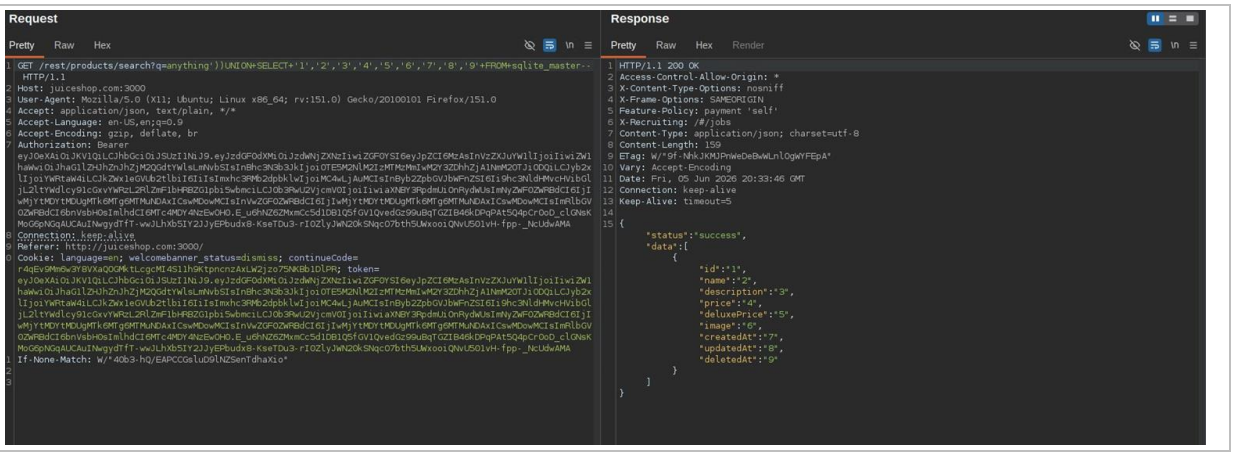


Figure 10c: Step 3 – Nine columns confirmed. Injecting nine placeholder values (UNION SELECT '1','2',…,'9' FROM sqlite_master--) returns 200 OK, and each placeholder is echoed back into a named JSON field (id, name, description, price … deletedAt). The query is now balanced, and those reflected fields are the channels used to pull data out.

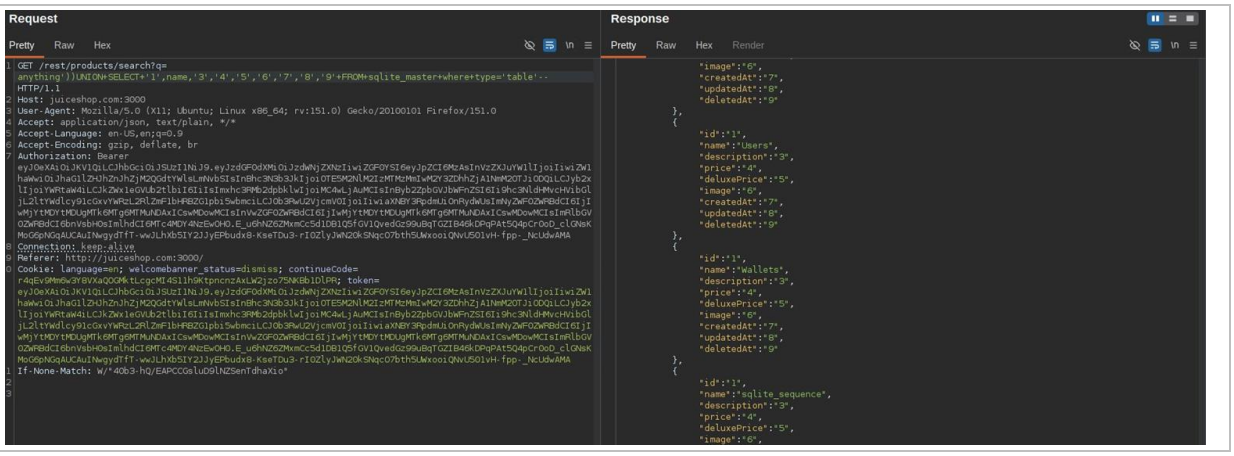


Figure 10d: Step 4 – Listing the tables. Replacing the second placeholder with the name column and querying

sqlite_master WHERE type='table' returns the database table names in the response. The Users table is clearly visible — the target for credential extraction.

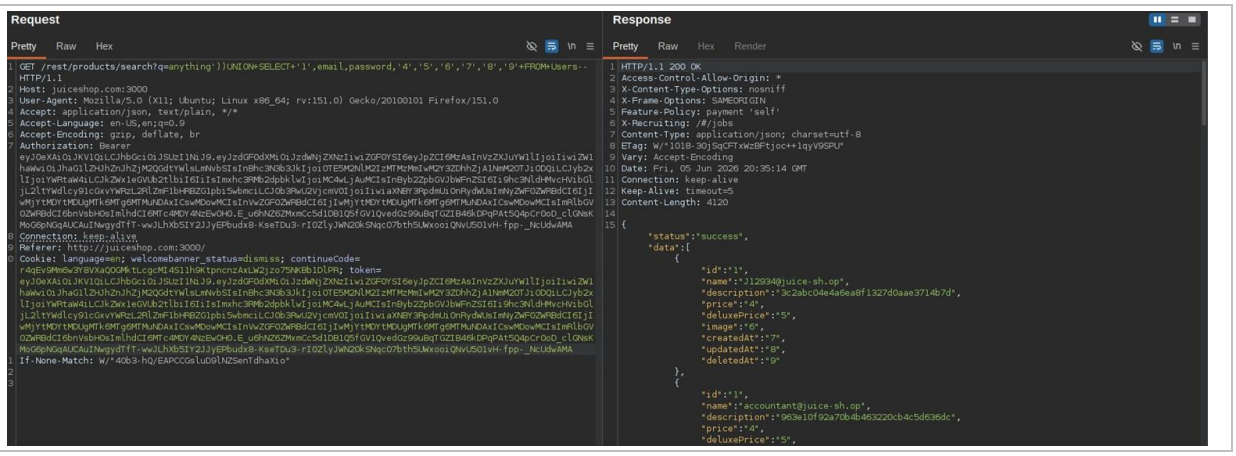


Figure 10e: Step 5 – Dumping the credentials. Selecting email and password from the Users table returns each account's email address and its password hash in the reflected fields. The password values are 32-character hexadecimal strings — the signature of MD5 (for example 0192023a7bbd73250516f069df18b500).

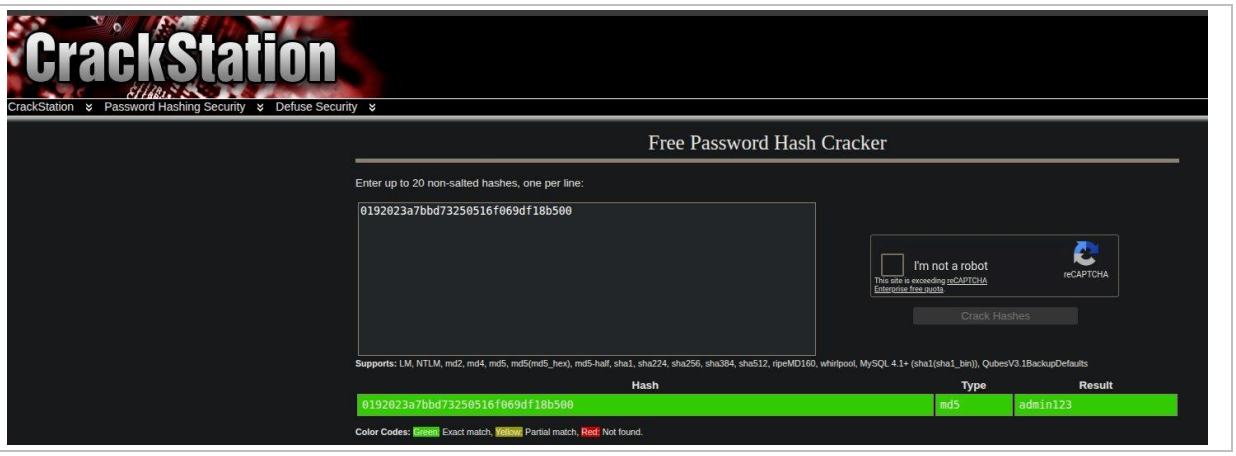


Figure 10f: Step 6 – Cracking the hash. Pasting a recovered hash into CrackStation identifies it as MD5 and instantly returns the plaintext password (admin123) from its lookup tables. Because MD5 is fast and the hashes are unsalted, recovery is effectively instant — the concrete impact of the weak storage scheme.

Remediation

Store passwords using a slow, salted, adaptive hashing function such as bcrypt, scrypt, or Argon2.

Migrate existing hashes on next login and enforce a strong password policy.

Finding WAPT-007: Mass Assignment – Privilege Escalation at Registration (High)

Description:	The user-registration endpoint trusts client-supplied JSON and does not restrict which fields a user may set. By adding a privileged attribute to the registration request body, an attacker can self-provision an administrative account (mass assignment / over-posting). Request body manipulation: add **`"role":"admin"`** to POST /api/Users
OWASP Category:	A04:2021 – Insecure Design / A01:2021 – Broken Access Control
Risk:	**Likelihood:** High – Easily performed by tampering with the request in a proxy. **Impact:** High – Self-service creation of administrative accounts.
Affected Component:	POST /api/Users
Tools Used:	Burp Suite
References:	OWASP Top 10 2021 – A04 Insecure Design OWASP Mass Assignment Cheat Sheet

Evidence

User registration sends a JSON body to POST /api/Users. The application binds the whole body to the new user record without restricting which fields a client may set, so an extra role field added to the request is accepted and stored — turning an ordinary sign-up into an administrator account.

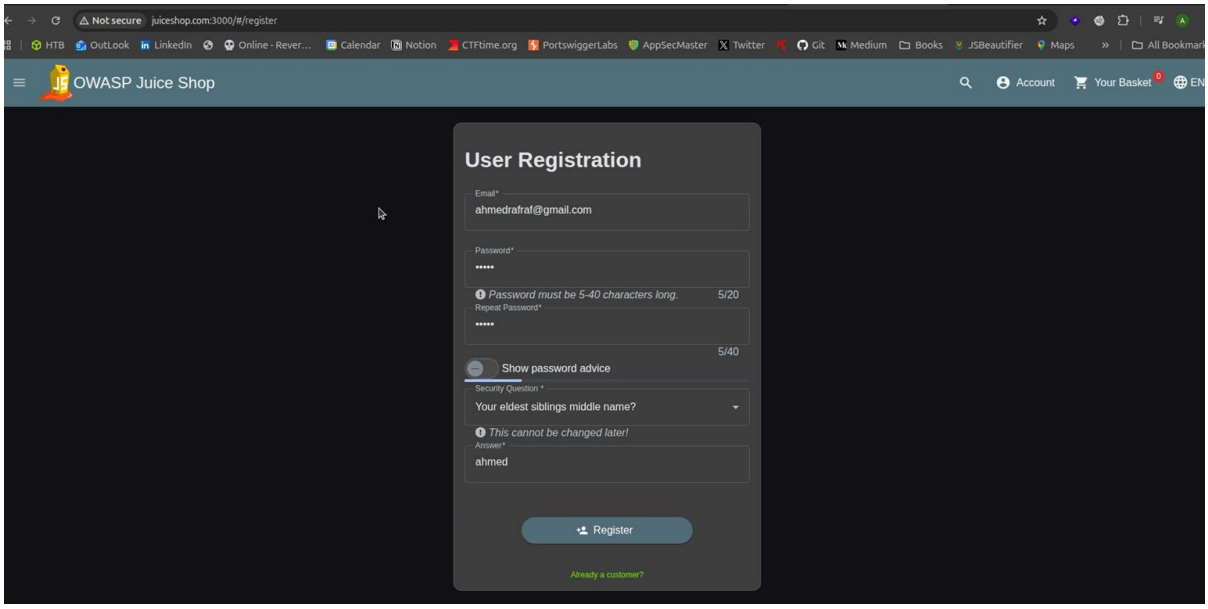


Figure 11a: Step 1 – A normal registration. The public User Registration form is completed with an email, password and security question. Submitting it sends a POST /api/Users request, which can be intercepted and modified in a proxy before it reaches the server.

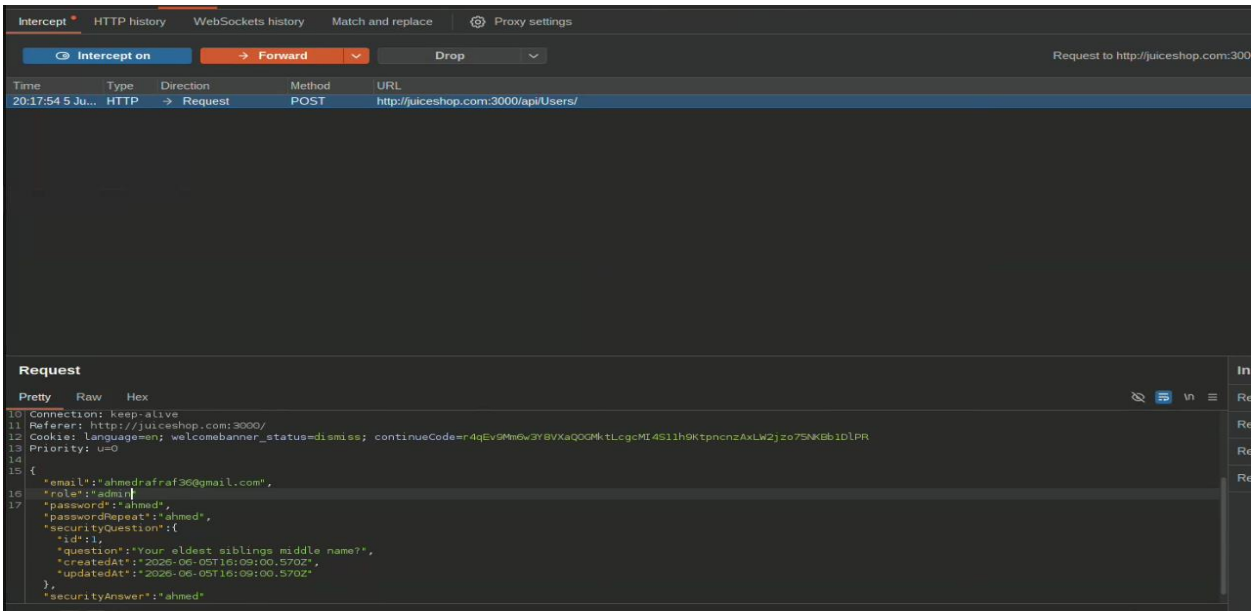


Figure 11b: Step 2 – The registration request and its result. The registration request to POST /api/Users (with a privileged role field added to the JSON body) is sent through Burp Suite. The server replies 201 Created and the returned user object shows role:admin and id 30 — confirming the server accepted a client-supplied role and provisioned an administrator account rather than a normal customer.

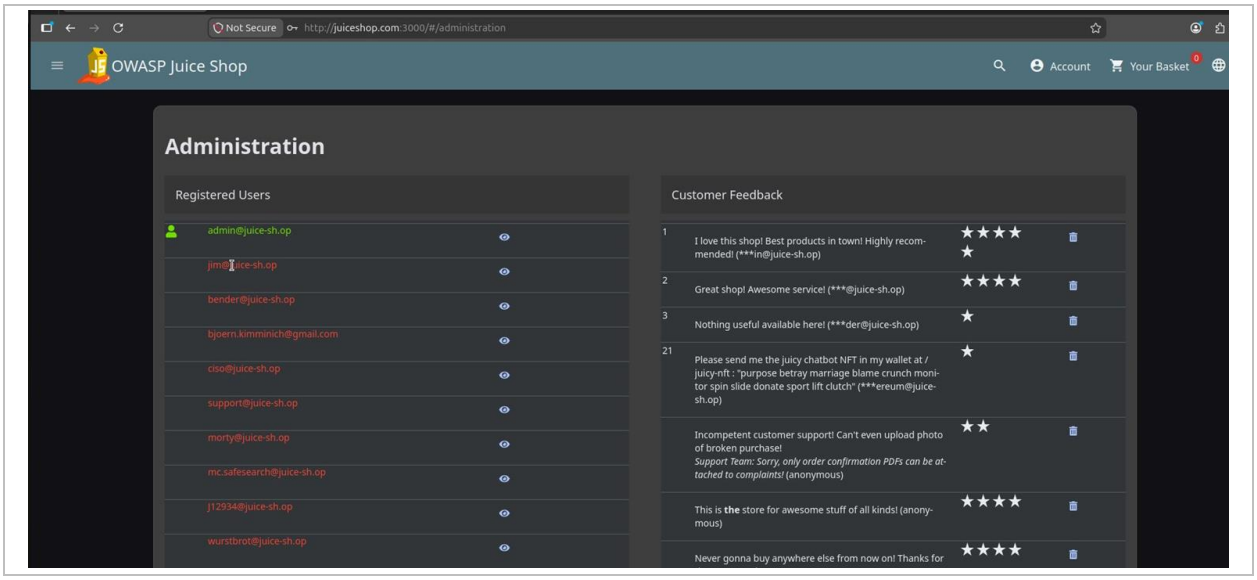


Figure 11c: Step 3 – Confirming the privileges. Signing in with the newly created account and opening /administration loads the admin-only panel, which lists every registered user and all customer feedback. A self-service sign-up has become a full administrator.

Remediation

Define a server-side allow-list of bindable properties; never accept security-sensitive fields such as role from the client.

Assign roles and privileges exclusively through trusted server-side logic.

Finding WAPT-008: Security Misconfiguration – Verbose Errors & Information Disclosure (Moderate)

Description:	The application returns detailed error messages, including SQL fragments and stack traces, and exposes operational endpoints that disclose internal information. This assists an attacker during enumeration and exploitation. Examples: verbose error responses, **`/metrics`** (Prometheus data), **`/rest/admin/application-version`**
OWASP Category:	A05:2021 – Security Misconfiguration
Risk:	**Likelihood:** Moderate – Information is returned by the application during normal interaction. **Impact:** Moderate – Leaked technical detail accelerates other attacks.
Affected Component:	Application error handling, /metrics endpoint
Tools Used:	Web browser, Burp Suite
References:	OWASP Top 10 2021 – A05 Security Misconfiguration OWASP WSTG – Testing for Error Handling (WSTG-ERRH)

Evidence

This finding groups two issues. Verbose error messages were already observed in earlier tests (the SQLite stack trace in WAPT-001 and the 403 filter error in WAPT-005). In addition, operational endpoints are reachable without authentication and disclose internal information, as shown below.

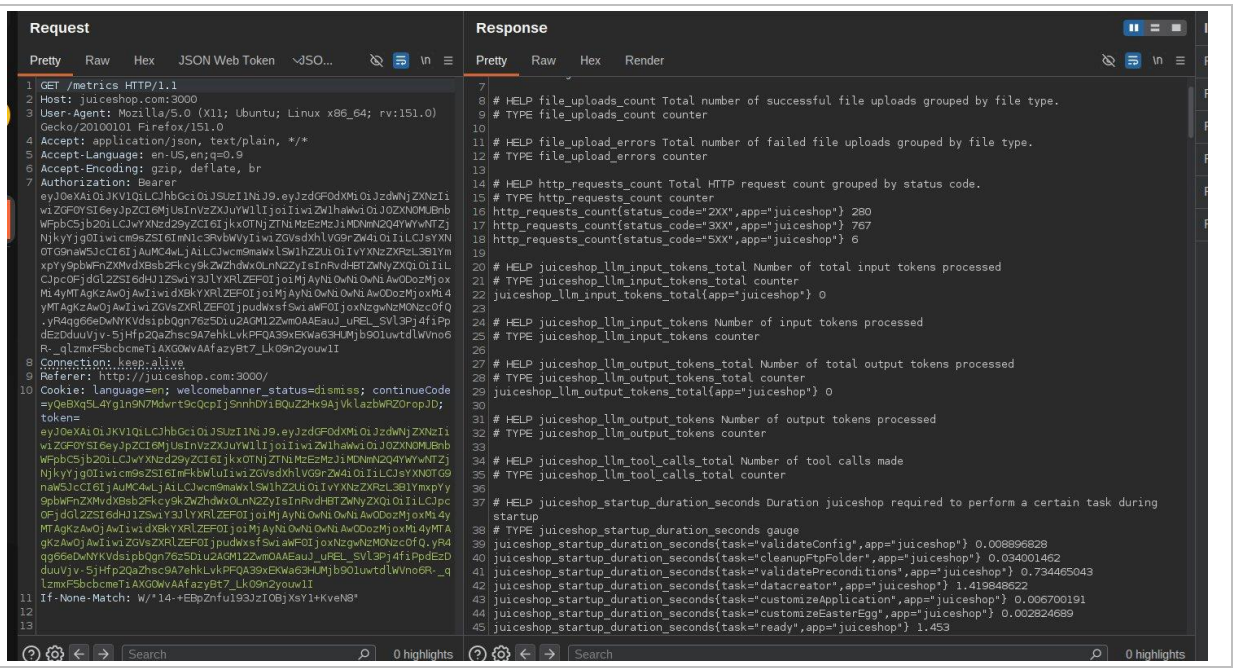


Figure 12a: Step 1 – Exposed metrics endpoint. GET /metrics returns Prometheus telemetry with no authentication. The dump includes HTTP request counts by status code, file-upload counters, LLM token and tool-call counters, and

startup-task timings (juiceshop_startup_duration_seconds) — internal runtime detail that helps an attacker profile the application.

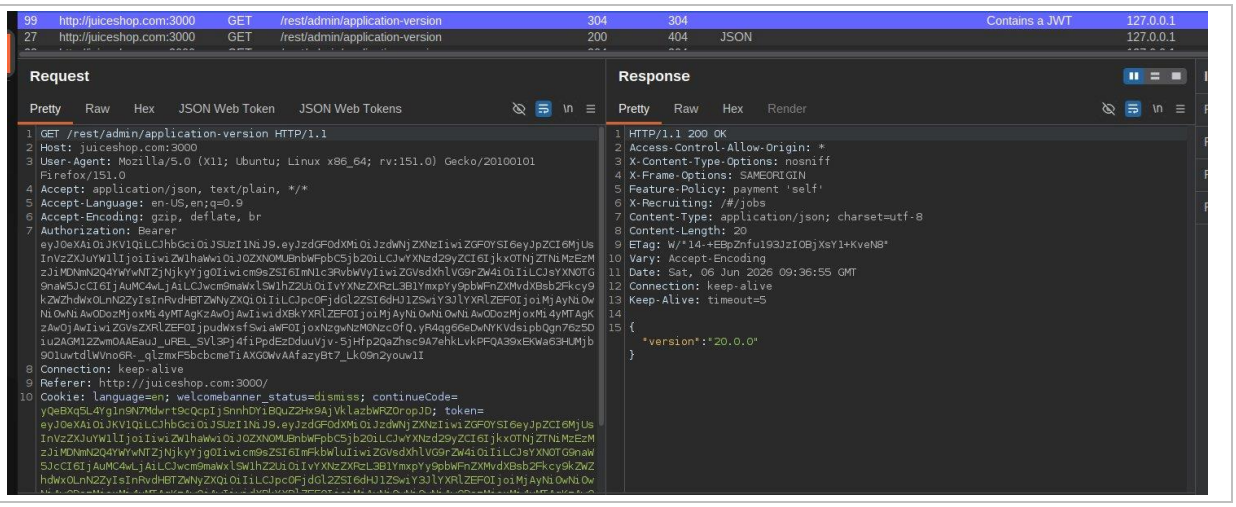


Figure 12b: Step 2 – Exposed application version. GET /rest/admin/application-version returns the JSON {"version":"20.0.0"} without authentication. Disclosing the exact version lets an attacker look up known vulnerabilities and public exploits for that specific release.

Remediation

Return generic error messages to clients, disable debug output in production, and log details server-side only.

Restrict or authenticate operational endpoints such as metrics and version information.

Appendix A – Methodology & Tools

The assessment was performed on Kali Linux against a local Juice Shop instance. The following tools were used across the engagement:

- Interception & manipulation: Burp Suite, browser
- Injection: manual payloads, sqlmap
- Password cracking: Crackstation ,Hashcat, John the Ripper